

DULCEBYTE

FASE 4 - APLICACIÓN MÓVIL CON FLUTTER

Evidencias de desarrollo e integración móvil

Aplicación móvil conectada a la API REST de DulceByte



Objetivo de la fase

Desarrollar una aplicación móvil que consuma la API REST creada en la Fase 2, permita consultar información del sistema, registrar al menos una operación de escritura, manejar estados de carga y errores de conexión, y ofrecer navegación entre pantallas.

Tecnología	Flutter / Dart
Consulta	GET /api/productos y GET /api/clientes
Escritura	POST /api/pedidos
Navegación	Inicio, Productos y Pedido
Carga y errores	Indicadores de carga, timeout, mensajes y reintento
Identidad visual	Paleta DulceByte consistente con la aplicación web

1. Estructura y arquitectura del proyecto

El proyecto se organizó por responsabilidades para mantener una estructura clara y fácil de mantener. La interfaz, los modelos de datos, la configuración de la API y los servicios HTTP se encuentran separados.

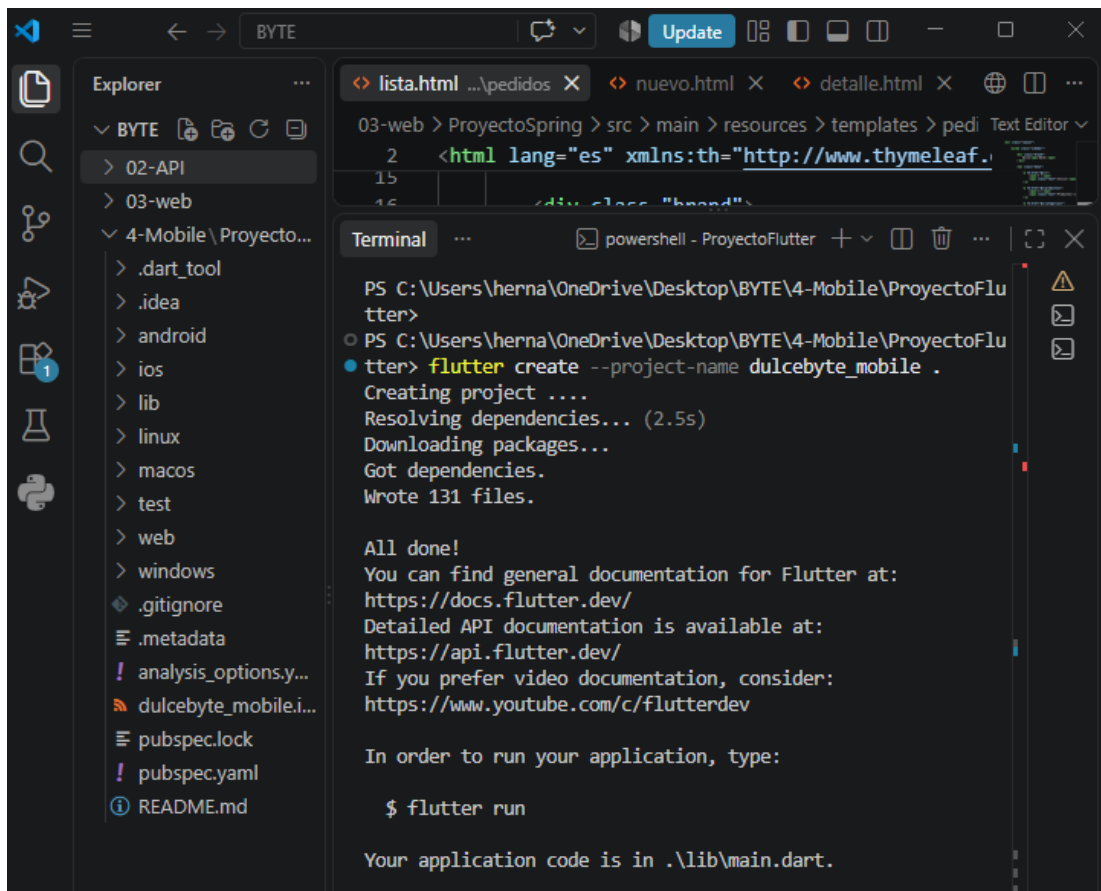


Figura 1. Proyecto Flutter generado y preparado para el desarrollo de la Fase 4.

Estructura principal de lib/

```
lib/  
  main.dart  
  config/api_config.dart  
  models/cliente.dart  
  models/producto.dart  
  models/pedido.dart  
  services/producto_service.dart  
  services/pedido_service.dart  
  screens/home_screen.dart  
  screens/productos_screen.dart  
  screens/nuevo_pedido_screen.dart  
  widgets/producto_card.dart
```

La separación por capas mantiene la configuración, los datos, las solicitudes HTTP y la interfaz en componentes independientes. Esto facilita la lectura del código y evita concentrar toda la lógica en main.dart.

2. Configuración y conexión con la API REST

La aplicación no accede directamente a MySQL. Toda la información se consulta y registra a través de la API REST de DulceByte. Esto mantiene separada la interfaz móvil de la lógica y persistencia del sistema.

Flutter Web / Windows	http://localhost:5080
Emulador Android	http://10.0.2.2:5080
Variable opcional	--dart-define=API_BASE_URL=...
Tiempo máximo de espera	10 segundos por solicitud

Operaciones consumidas

Método	Endpoint	Uso en Flutter
GET	/api/productos	Consultar catálogo de productos
GET	/api/clientes	Cargar clientes para el pedido
POST	/api/pedidos	Registrar un nuevo pedido

Productos

GET /api/productos

Parameters

Name

Description

idCategoria

integer(\$int32)

(query)

1

disponible

boolean

(query)

true

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:5080/api/productos?idCategoria=1&disponible=true' \
-H 'accept: text/plain'
```

Request URL

```
http://localhost:5080/api/productos?idCategoria=1&disponible=true
```

Server response

Figura 2. Endpoint GET /api/productos disponible en la API REST.

3. Interfaz móvil y navegación

La pantalla principal ofrece accesos directos a las dos funciones implementadas en la aplicación móvil. La navegación inferior permite cambiar entre Inicio, Productos y Pedido sin abandonar la aplicación.



Figura 3. Pantalla de inicio de DulceByte Mobile con accesos a Productos y Pedidos.

Pantalla	Función
Inicio	Accesos rápidos y descripción de las funciones móviles.
Productos	Consulta del catálogo actualizado mediante la API REST.
Pedido	Selección de cliente y registro de un nuevo pedido.

4. Consulta de información y manejo de estados

La pantalla Productos realiza una solicitud GET al endpoint `/api/productos`. Mientras espera la respuesta muestra un indicador de carga. Si la solicitud falla, se muestra un mensaje de error y se habilita una opción para reintentar.

Estado	Comportamiento
Cargando	CircularProgressIndicator mientras se espera la respuesta.
Éxito	Lista de tarjetas con la información de los productos.
Sin datos	Mensaje indicando que no existen productos registrados.
Error	Mensaje de conexión y botón Reintentar.

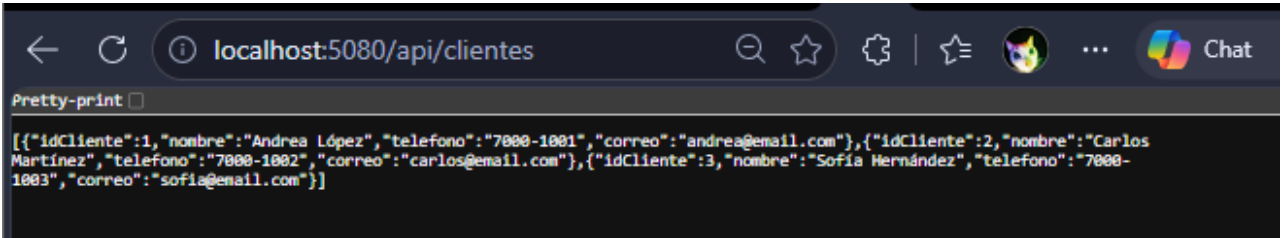


Figura 4. Respuesta del endpoint `GET /api/clientes` utilizada para llenar el selector de clientes.

Los servicios también controlan respuestas con formato inválido, tiempos de espera y fallos de conexión. Los errores se transforman en mensajes comprensibles para el usuario en lugar de mostrar excepciones técnicas.

5. Operación de escritura: registro de pedido

Para cumplir la operación de escritura requerida, la aplicación permite seleccionar un cliente y registrar un pedido mediante POST /api/pedidos. El cuerpo enviado contiene el identificador interno del cliente seleccionado.

```
POST /api/pedidos
Content-Type: application/json

{
  "idCliente": <cliente seleccionado>
}
```

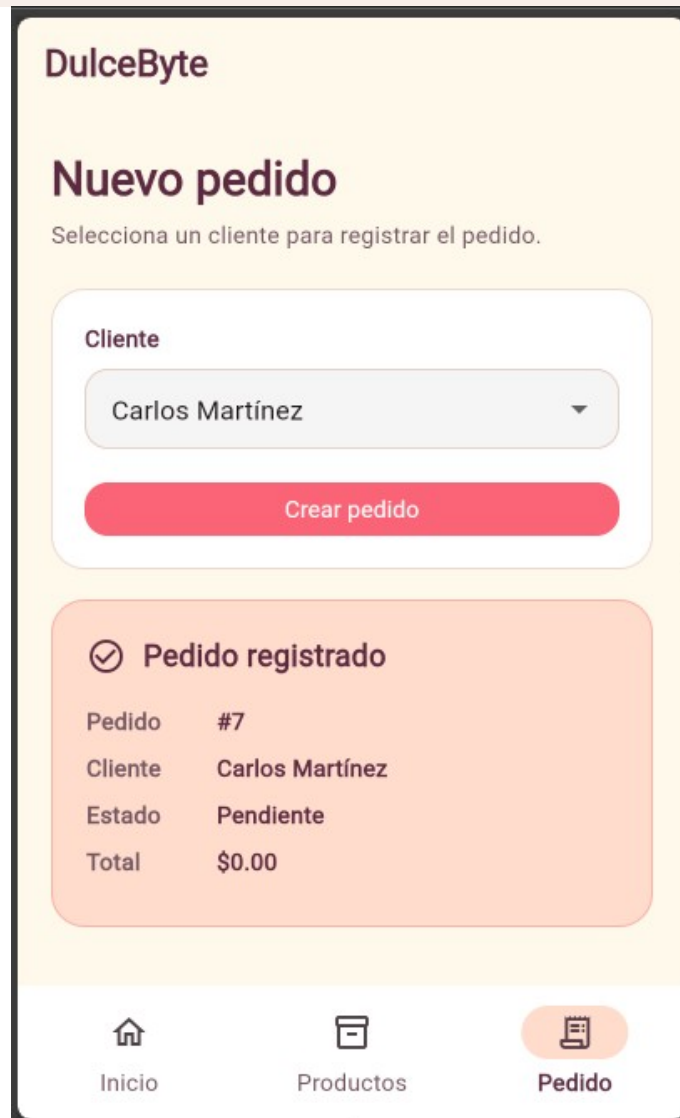


Figura 5. Pedido #7 registrado correctamente desde la aplicación Flutter para Carlos Martínez.

La respuesta de la API se transforma en un objeto Pedido y se presenta en una tarjeta de confirmación con número de pedido, cliente, estado inicial y total. En la evidencia se observa el estado Pendiente y total \$0.00, correspondientes a un pedido recién creado sin detalles agregados.

6. Verificación de requisitos de la Fase 4

Requisito	Implementación	Estado
Conectarse a la API	Configuración centralizada mediante ApiConfig y paquete http.	Cumplido
Consultar información	GET /api/productos y GET /api/clientes.	Cumplido
Mostrar información móvil	Pantallas Material 3 y tarjetas adaptadas a viewport móvil.	Cumplido
Operación de escritura	POST /api/pedidos desde NuevoPedidoScreen.	Cumplido
Estados de carga	CircularProgressIndicator en consultas y envío.	Cumplido
Errores de conexión	Timeout, mensajes de error y opción Reintentar.	Cumplido
Navegación	NavigationBar entre Inicio, Productos y Pedido.	Cumplido

Conclusión

La Fase 4 integra correctamente una interfaz móvil Flutter con la API REST desarrollada anteriormente. La aplicación consulta información real del sistema, registra pedidos, administra estados de carga y fallos de conexión, y mantiene una navegación e identidad visual coherentes con DulceByte. Con ello se cubren los requisitos funcionales establecidos para la aplicación móvil.

Estructura de entrega

04-Mobile/ ProyectoFlutter/ EvidenciasMobile.pdf
--